

Harvinder S Jabbal
CSIS, Work Integrated Learning
Programs



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Software Architecture CS01 Introduction

July 25, 2026

SEZG651/SSZG653 Software
Architectures



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Why Software Architecture is Important?

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs

July 25, 2026

SEZG651/SSZG653 Software
Architectures



SEZG651/ SSZG653 Software Architectures CS01

July 25, 2026

SEZG651/SSZG653 Software
Architectures

Why is Software Architecture Important?



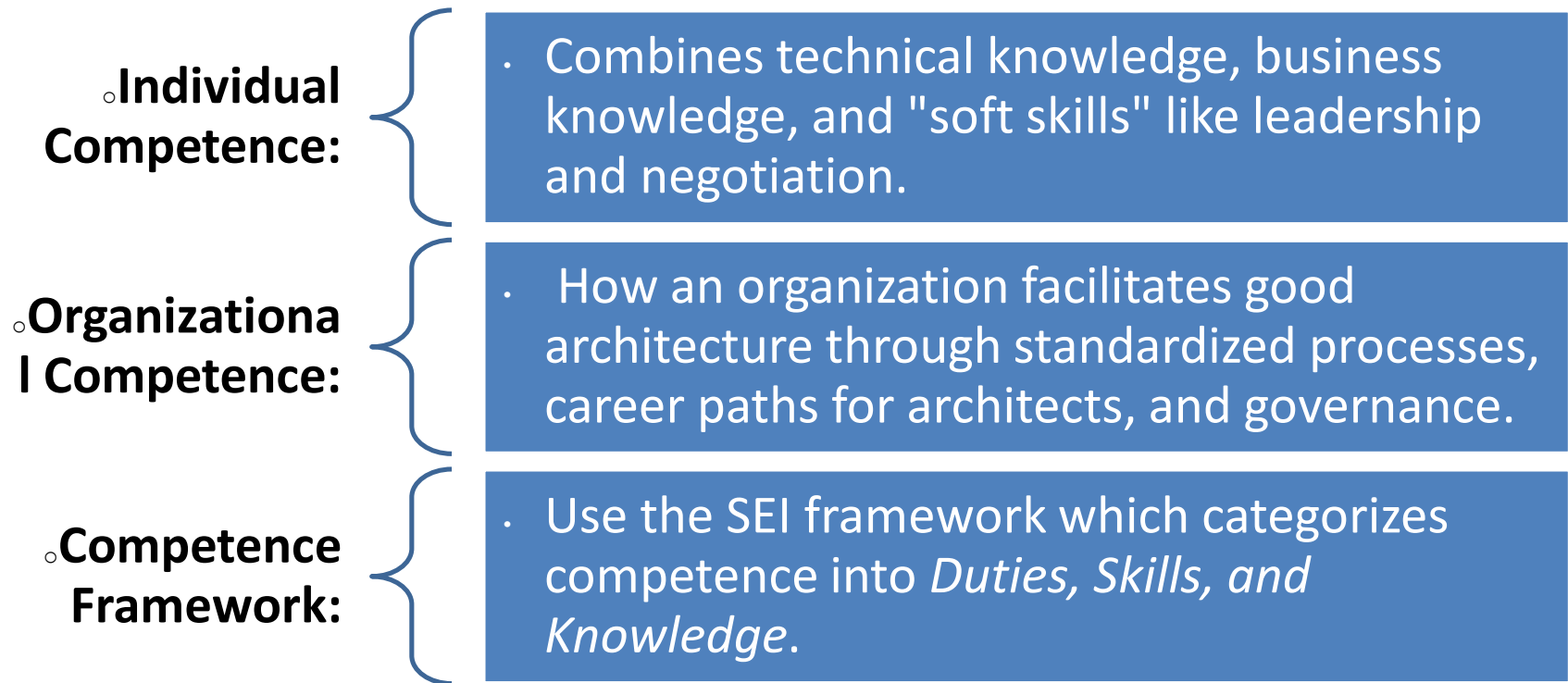
1. An architecture will inhibit or enable a system's driving quality attributes.
2. The decisions made in an architecture allow you to reason about and manage change as the system evolves.
3. The analysis of an architecture enables early prediction of a system's qualities.
4. A documented architecture enhances communication among stakeholders.
5. The architecture is a carrier of the earliest and hence most fundamental, hardest-to-change design decisions.
6. An architecture defines a set of constraints on subsequent implementation.
7. The architecture dictates the structure of an organization, or vice versa.
8. An architecture can provide the basis for evolutionary prototyping.
9. An architecture is the key artifact that allows the architect and project manager to reason about cost and schedule.
10. An architecture can be created as a transferable, reusable model that form the heart of a product line.
11. Architecture-based development focuses attention on the assembly of components, rather than simply on their creation.
12. By restricting design alternatives, architecture channels the creativity of developers, reducing design and system complexity.
13. An architecture can be the foundation for training a new team member.

As per Len Bass



- 1. Inhibiting or enabling quality attributes.
- 1. Reasoning about and managing change.
- 1. Predicting system qualities.
- 1. Communication among stakeholders.
- 1. Carrying early design decisions.
- 1. Defining constraints on implementation.
- 1. Influencing organizational structure (Conway's Law).
- 1. Enabling incremental development.
- 1. Improving cost and schedule estimates.
- 1. Supplying a transferable, reusable model.
- 1. Allowing incorporation of independently developed elements.
- 1. Restricting the vocabulary of design alternatives.
- 1. Providing a basis for training.

Architecture Competence



Inhibiting or Enabling a System's Quality Attributes



Whether a system will be able to exhibit its desired (or required) quality attributes is substantially determined by its architecture.

This is the most important message of this course!

- Performance: You must manage the time-based behavior of elements, their use of shared resources, and the frequency and volume of inter-element communication.
- Modifiability: Assign responsibilities to elements so that the majority of changes to the system will affect a small number of those elements.
- Security: Manage and protect inter-element communication and control which elements are allowed to access which information; you may also need to introduce specialized elements (such as an authorization mechanism).
- Scalability: Localize the use of resources to facilitate introduction of higher-capacity replacements, and you must avoid hardcoding in resource assumptions or limits.
- Incremental subset delivery: Manage inter-component usage.
- Reusability: Restrict inter-element coupling, so that when you extract an element, it does not come out with too many attachments to its current environment.

Reasoning About and Managing Change



- About 80 percent of a typical software system's total cost occurs after initial deployment
 - accommodate new features
 - adapt to new environments,
 - fix bugs, and so forth.
- Every architecture partitions possible changes into three categories
 - A *local* change can be accomplished by modifying a single element.
 - A *nonlocal* change requires multiple element modifications but leaves the underlying architectural approach intact.
 - An *architectural* change affects the fundamental ways in which the elements interact with each other and will probably require changes all over the system.
- Obviously, local changes are the most desirable
- A good architecture is one in which the most common changes are local, and hence easy to make.

Predicting System Qualities



- If we know that certain kinds of architectural decisions lead to certain quality attributes in a system, we can make those decisions and rightly expect to be rewarded with the associated quality attributes.
- When we examine an architecture we can look to see if those decisions have been made, and confidently predict that the architecture will exhibit the associated qualities.
- The earlier you can find a problem in your design, the cheaper, easier, and less disruptive it will be to fix.

Enhancing Communication Among Stakeholders



- Software architecture represents a common abstraction of a system that most, if not all, of the system's stakeholders can use as a basis for creating mutual understanding, negotiating, forming consensus, and communicating with each other.
- The architecture—or at least parts of it—is sufficiently abstract that most nontechnical people can understand it adequately.
- Each stakeholder of a software system—customer, user, project manager, coder, tester, and so on—is concerned with different characteristics of the system that are affected by its architecture. For example:
 - The user is concerned that the system is fast, reliable, and available when needed.
 - The customer is concerned that the architecture can be implemented on schedule and according to budget.
 - The manager is worried (in addition to concerns about cost and schedule) that the architecture will allow teams to work largely independently, interacting in disciplined and controlled ways.
 - The architect is worried about strategies to achieve all of those goals.
- Architecture provides a common language in which different concerns can be expressed, negotiated, and resolved at a level that is intellectually manageable even for large, complex systems.

Earliest Design Decisions



- Software architecture is a manifestation of the earliest design decisions about a system.
- These early bindings carry enormous weight with respect to the system's remaining development, its deployment, and its maintenance life.
- Each decision constrains the many decisions that follow.
- What are these early design decisions embodied by software architecture?
 - Will the system run on one processor or be distributed across multiple processors?
 - Will the software be layered? If so, how many layers will there be? What will each one do?
 - Will components communicate synchronously or asynchronously? Will they interact by transferring control or data or both?
 - Will the system depend on specific features of the operating system or hardware?
 - Will the information that flows through the system be encrypted or not?
 - What communication protocol will we choose?
- Imagine the nightmare of having to change any of these decisions.

Defining Constraints on an Implementation



- An implementation exhibits an architecture if it conforms to the design decisions prescribed by the architecture.
 - The implementation must be implemented as the set of prescribed elements
 - These elements must interact with each other in the prescribed fashion
 - Each element must fulfill its responsibility to the other elements as dictated by the architecture.
- Each of these prescriptions is a constraint on the implementer.
- Element builders may not be aware of the architectural tradeoffs—the architecture (or architect) simply constrains them in such a way as to meet the tradeoffs.
 - Example: an architect assigns performance budget to the pieces of software involved in some larger piece of functionality.
 - If each software unit stays within its budget, the overall transaction will meet its performance requirement.
 - Implementers of each of the constituent pieces may not know the overall budget, only their own.

Influencing the Organizational Structure



- Architecture prescribes the structure of the system being developed.
- That structure becomes engraved in the structure of the development project (and sometimes the structure of the entire organization).
- The architecture is typically used as the basis for the work-breakdown structure.
- The work-breakdown structure in turn dictates
 - units of planning, scheduling, and budget
 - interteam communication channels
 - configuration control and file-system organization
 - integration and test plans and procedures;
 - much more
- The maintenance activity will also reflect the software structure, with teams formed to maintain specific structural elements from the architecture.
- If these responsibilities have been formalized in a contractual relationship, changing responsibilities could become expensive or even litigious.

Enabling Evolutionary Prototyping



- Once an architecture has been defined, it can be analyzed and prototyped as a skeletal system.
 - A skeletal system is one in which at least some of the infrastructure— how the elements initialize, communicate, share data, access resources, report errors, log activity, and so forth—is built before much of the system’s functionality has been created.
- This approach aids the development process because the system is executable early in the product’s life cycle.
- The fidelity of the system increases as stubs are instantiated, or prototype parts are replaced with complete versions of these parts of the software.
- This approach allows potential performance problems to be identified early in the product’s life cycle.
- These benefits reduce the potential risk in the project.

Improving Cost and Schedule Estimates



- One of the duties of an architect is to help the project manager create cost and schedule estimates early in the project life cycle.
- Top-down estimates are useful for setting goals and apportioning budgets.
- Cost estimations that are based on a bottom-up understanding of the system's pieces are typically more accurate than those that are based purely on top-down system knowledge.
 - Each team or individual responsible for a work item will be able to make more-accurate estimates for their piece than a project manager and will feel more ownership in making the estimates come true.
- The best cost and schedule estimates will typically emerge from a consensus between the top-down estimates (created by the architect and project manager) and the bottom-up estimates (created by the developers).

Transferable, Reusable Model



- Reuse of architectures provides tremendous leverage for systems with similar requirements.
 - Not only can code be reused, but so can the requirements that led to the architecture in the first place, as well as the experience and infrastructure gained in building the reused architecture.
 - When architectural decisions can be reused across multiple systems, all of the early-decision consequences are also transferred.
- A software product line or family is a set of software systems that are all built using the same set of reusable assets.
 - Chief among these assets is the architecture that was designed to handle the needs of the entire family.
 - The architecture defines what is fixed for all members of the product line and what is variable.
 - The architecture for a product line becomes a capital investment by the organization.

Using Independently Developed Components



- Architecture-based development often focuses on components that are likely to have been developed separately, even independently, from each other.
- The architecture defines the elements that can be incorporated into the system.
- Commercial off-the-shelf components, open source software, publicly available apps, and networked services are example of interchangeable software components.
- The payoff can be
 - Decreased time to market
 - Increased reliability (widely used software should have its bugs ironed out already)
 - Lower cost (the software supplier can amortize development cost across their customer base)
 - Flexibility (if the component you want to buy is not terribly specialpurpose, it's likely to be available from several sources, thus increasing your buying leverage)

Restricting Design Vocabulary



- As useful architectural patterns are collected, we see the benefit in voluntarily restricting ourselves to a relatively small number of choices of elements and their interactions.
 - We minimize the design complexity of the system we are building.
 - Enhanced reuse
 - More regular and simpler designs that are more easily understood and communicated
 - More capable analysis
 - Shorter selection time
 - Greater interoperability.
- Architectural patterns guide the architect and focus the architect on the quality attributes of interest in large part by restricting the vocabulary of design.
 - Properties of software design follow from the choice of an architectural pattern.

Basis for Training



- The architecture can serve as the first introduction to the system for new project members.
- Module views are excellent for showing someone the structure of a project
 - Who does what, which teams are assigned to which parts of the system, and so forth.
- Component-and-connector views are excellent for explaining how the system is expected to work and accomplish its job.

Summary



1. An architecture will inhibit or enable a system's driving quality attributes.
2. The decisions made in an architecture allow you to reason about and manage change as the system evolves.
3. The analysis of an architecture enables early prediction of a system's qualities.
4. A documented architecture enhances communication among stakeholders.
5. The architecture is a carrier of the earliest and hence most fundamental, hardest-to-change design decisions.
6. An architecture defines a set of constraints on subsequent implementation.
7. The architecture dictates the structure of an organization, or vice versa.

Summary



8. An architecture can provide the basis for evolutionary prototyping.
9. An architecture is the key artifact that allows the architect and project manager to reason about cost and schedule.
10. An architecture can be created as a transferable, reusable model that form the heart of a product line.
11. Architecture-based development focuses attention on the assembly of components, rather than simply on their creation.
12. By restricting design alternatives, architecture channels the creativity of developers, reducing design and system complexity.
13. An architecture can be the foundation for training a new team member.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

The Many Contexts of Software Architecture

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs

Chapter Outline



Architecture in a Technical Context
Architecture in a Project Life-Cycle Context
Architecture in a Business Context
Architecture in a Professional Context
Stakeholders
How Is Architecture Influenced?
What Do Architectures Influence?
Summary

Contexts of Software Architecture



- Sometimes we consider software architecture the center of the universe!
- Here, though, we put it in its place relative to four contexts:
 - Technical. What technical role does the software architecture play in the system or systems of which it's a part?
 - Project life cycle. How does a software architecture relate to the other phases of a software development life cycle?
 - Business. How does the presence of a software architecture affect an organization's business environment?
 - Professional. What is the role of a software architect in an organization or a development project?

Technical Context



- The most important technical context factor is the set of quality attributes that the architecture can help to achieve.
- The architecture's current technical environment is also an important factor.
 - Standard industry practices
 - Software engineering techniques prevalent in the architect's professional community.
- Today's information systems are web-based, object-oriented, service-oriented, mobility-aware, cloud-based, social-networking-friendly.
 - It wasn't always so.
 - It won't be so ten years from now.

Project Life-cycle Context



- Software development processes are standard approaches for developing software systems.
- They impose a discipline on software engineers and, more important, teams of software engineers.
- They tell the members of the team what to do next.
- There are four dominant software development processes:
 - Waterfall
 - Iterative
 - Agile
 - Model-driven development

Architecture Activities



- All of these processes include design among their obligations.
- Architecture is a special kind of design, so architecture finds a home in each one.
- No matter the software development process, there are activities involved in creating a software architecture, using that architecture to realize a complete design, and then implementing or managing the evolution of a target system or application:
 - 1. Making a business case for the system
 - 2. Understanding the architecturally significant requirements
 - 3. Creating or selecting the architecture
 - 4. Documenting and communicating the architecture
 - 5. Analyzing or evaluating the architecture
 - 6. Implementing and testing the system based on the architecture
 - 7. Ensuring that the implementation conforms to the architecture

Business Context



Architectures and systems are not constructed frivolously.
They serve some business purposes.
These purposes may change over time.

Architecture and Business Goals



- Systems are created to satisfy the business goals of one or more organizations.
 - Development organizations want to make a profit, or capture market, or stay in business, or help their customers do their jobs better, or keep their staff gainfully employed, or make their stockholders happy, or a little bit of each.
 - Customers have their own goals for acquiring a system, usually involving some aspect of making their lives easier or more productive. Other organizations involved in a project's life cycle, such as subcontractors or government regulatory agencies, have their own goals dealing with the system.
- Architects need to understand who the vested organizations are and what their goals are. Many of these goals will have a profound influence on the architecture.

Architecture and Business Goals



- Every quality attribute—such as a user-visible response time or platform flexibility or ironclad security or any of a dozen other needs—should originate from some higher purpose that can be described in terms of added value.
 - “Why do you want this system to have a really fast response time?”
 - This differentiate the product from its competition and let the developing organization capture market share.
- Some business goals will not show up in the form of requirements.
- Still other business goals have no effect on the architecture whatsoever.
 - A business goal to lower costs might be realized by asking employees to work from home, or turn the office thermostats down in the winter, or using less paper in the printers.

Architecture and business goals

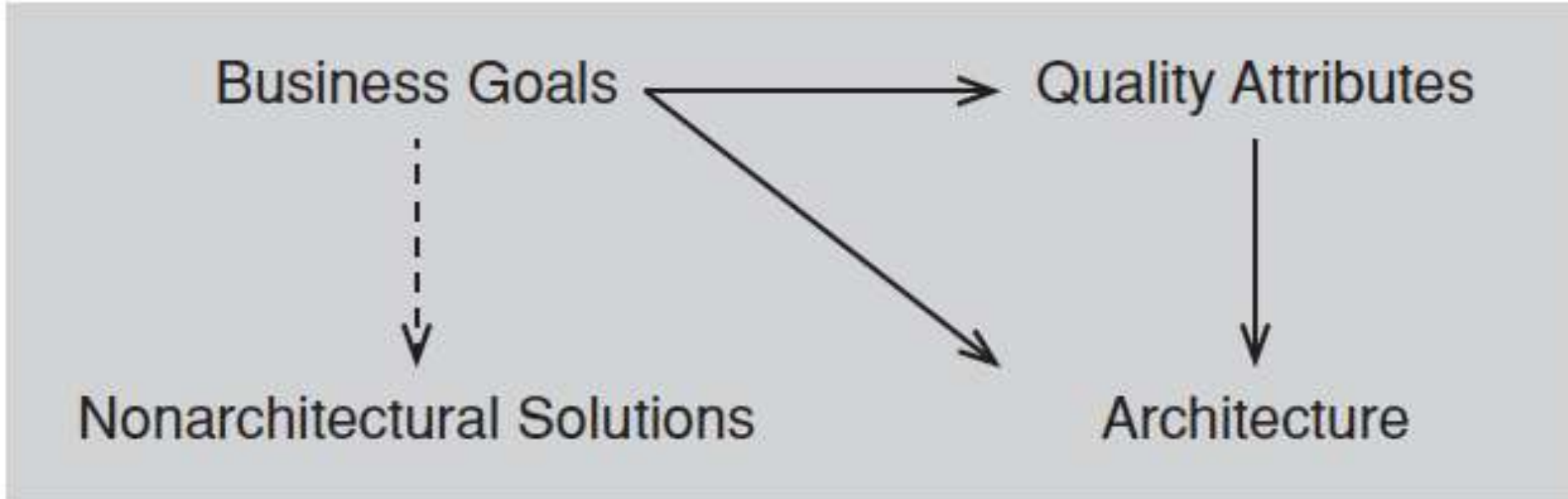


FIGURE 3.2 Some business goals may lead to quality attribute requirements (which lead to architectures), or lead directly to architectural decisions, or lead to nonarchitectural solutions.

Professional Context



You will perform many *duties* beyond directly producing an architecture.

- You will need to be involved in supporting management and dealing with customers.

Architects need more than just technical *skills*.

- Architects need to explain to one stakeholder or another the chosen priorities of different properties, and why particular stakeholders are not having all of their expectations fulfilled.
- Architects need diplomatic, negotiation, and communication skills.
- Architects need the ability to communicate ideas clearly
- You will need to manage a diverse workload and be able to switch contexts frequently.
- You will need to be a leader in the eyes of developers and management.

Architects need up-to-date *knowledge*.

- You will need to know about (for example) patterns, or database platforms, or web services standards.
- You will need to know business considerations.

Stakeholders



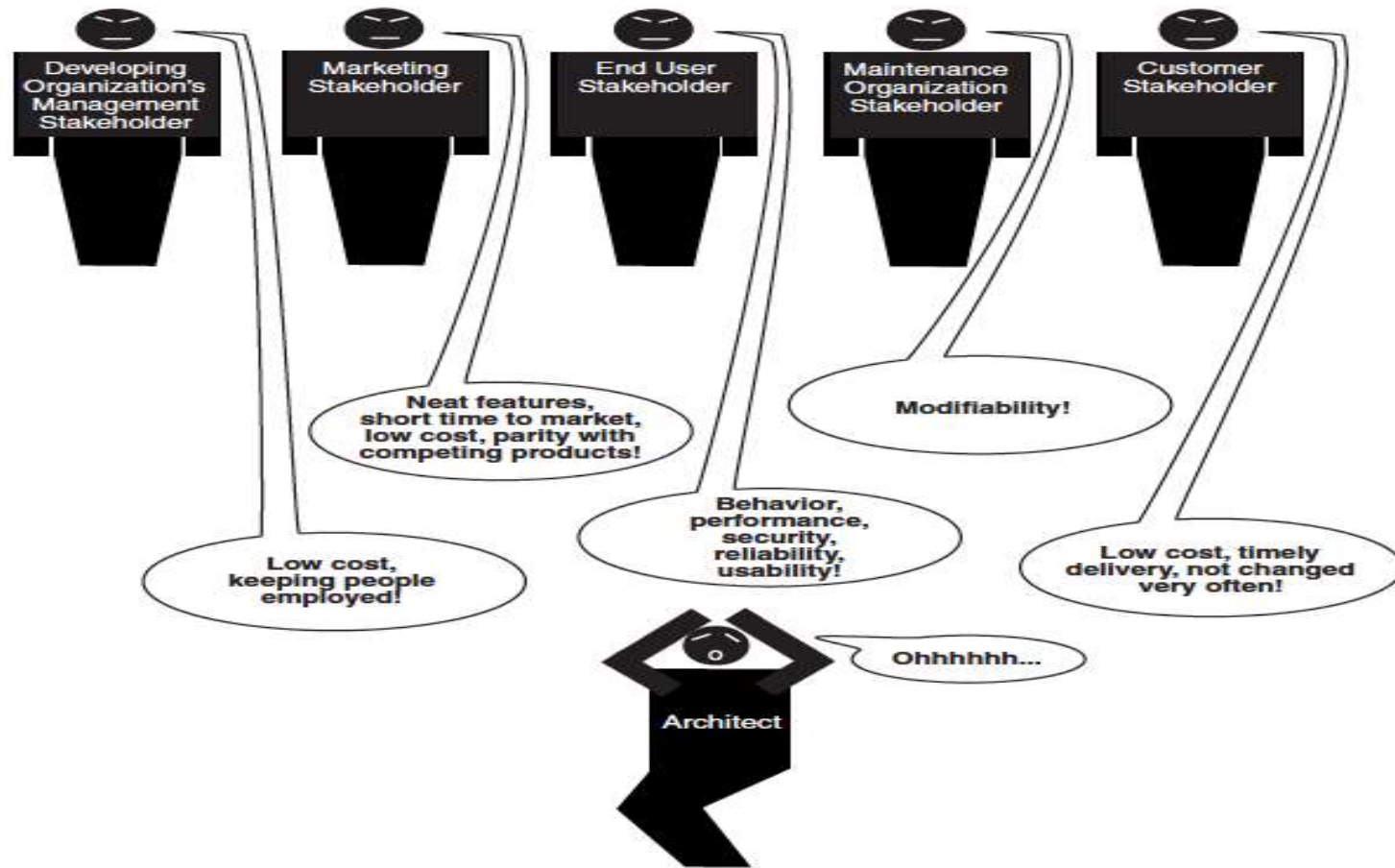
- A stakeholder is anyone who has a stake in the success of the system
- Stakeholders typically have different specific concerns that they wish the system to guarantee or optimize.
- You will need to know and understand the nature, source, and priority of constraints on the project as early as possible. Therefore, you must identify and actively engage the stakeholders to solicit their needs and expectations.
- Early engagement of stakeholders allows you to understand the constraints of the task, manage expectations, negotiate priorities, and make tradeoffs.

Stakeholders



- Know your stakeholders!
- Talk to them, engage them, listen to them, and put yourself in their shoes.
- See Table for a list of example stakeholders and their interests and concerns.

Stakeholders



How is Architecture Influenced?



- Requirements influence the architecture, of course.
- But the requirements specification only begins to tell the story.
- A software architecture is a result of business and social influences, as well as technical ones.
- The existence of an architecture in turn affects the technical, business, and social environments that subsequently influence future architectures.
- In particular, each of the contexts for architecture plays a role in influencing an architect and the architecture.

How is Architecture Influenced?

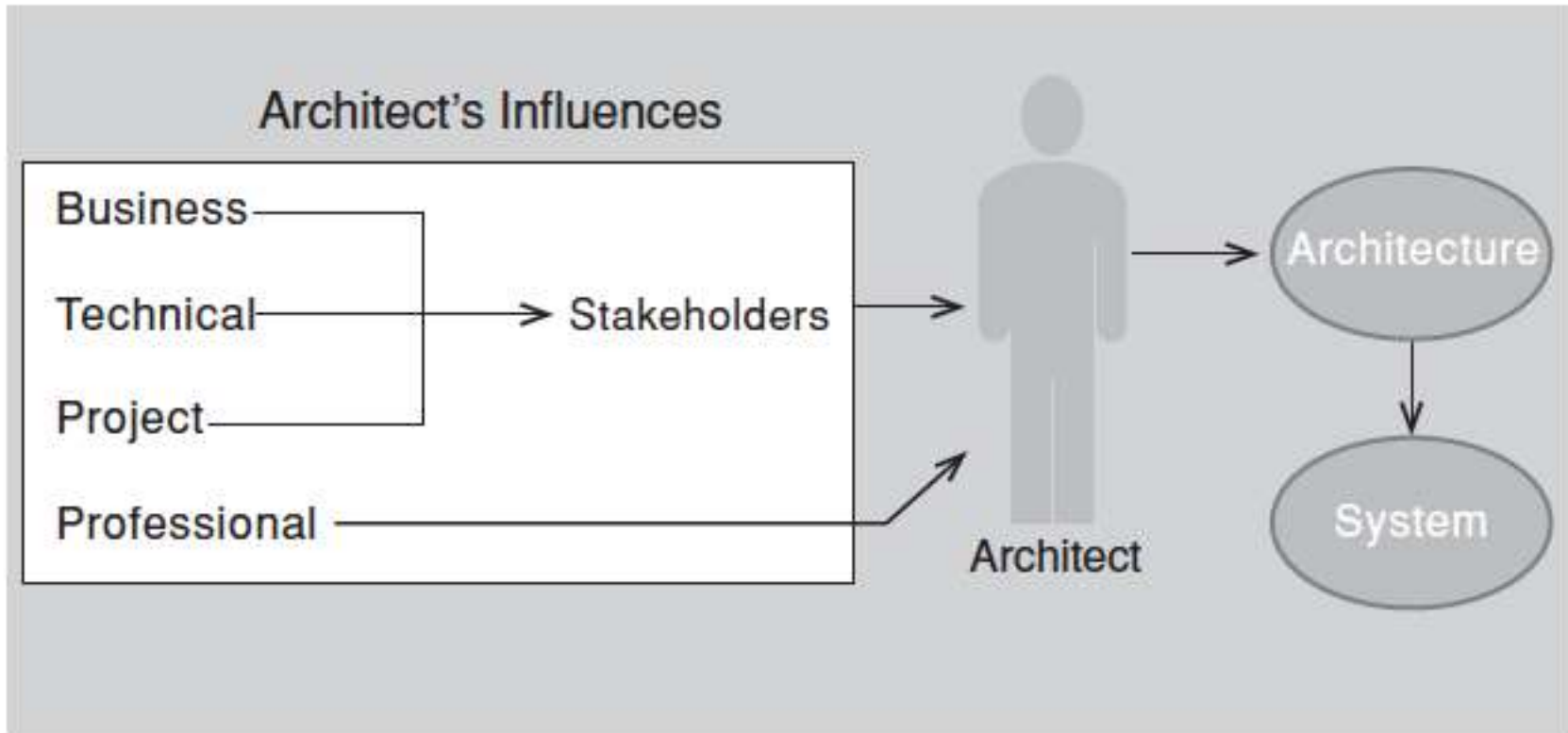


FIGURE 3.4 Influences on the architect

What Do Architectures Influence?



Technical context

- The architecture can affect stakeholder requirements for the next system
- It gives the customer the opportunity to receive a system (based on the same architecture) in a more reliable, timely, and economical manner than if built from scratch.
- A customer may in fact be willing to relax some of their requirements to gain these economies.
- Shrinkwrapped software has clearly affected people's requirements by providing solutions that are not tailored to any individual's precise needs but are instead inexpensive and (in the best of all possible worlds) of high quality.

What Do Architectures Influence?



Project context

- The architecture affects the structure of the developing organization.
- An architecture prescribes the units of software that must be implemented (or otherwise obtained) and integrated to form the system.
- These units are the basis for the development project's structure.
- Teams are formed for individual software units; and the development, test, and integration activities all revolve around the units.
- Teams become embedded in the organization's structure.

What Do Architectures Influence?



Business context

- The architecture can affect the business goals of the developing organization.
- A successful system built from an architecture can enable a company to establish a foothold in a particular market segment.
- The architecture can provide opportunities for the efficient production and deployment of similar systems, and the organization may adjust its goals to take advantage of its newfound expertise to plumb the market.

What Do Architectures Influence?



Professional context

- The process of system building will affect the architect's experience with subsequent.
- A system that was successfully built around a particular technical approach will make the architect more inclined to build systems using the same approach in the future.
- Architectures that fail are less likely to be chosen for future projects.

Architecture Influence Cycle

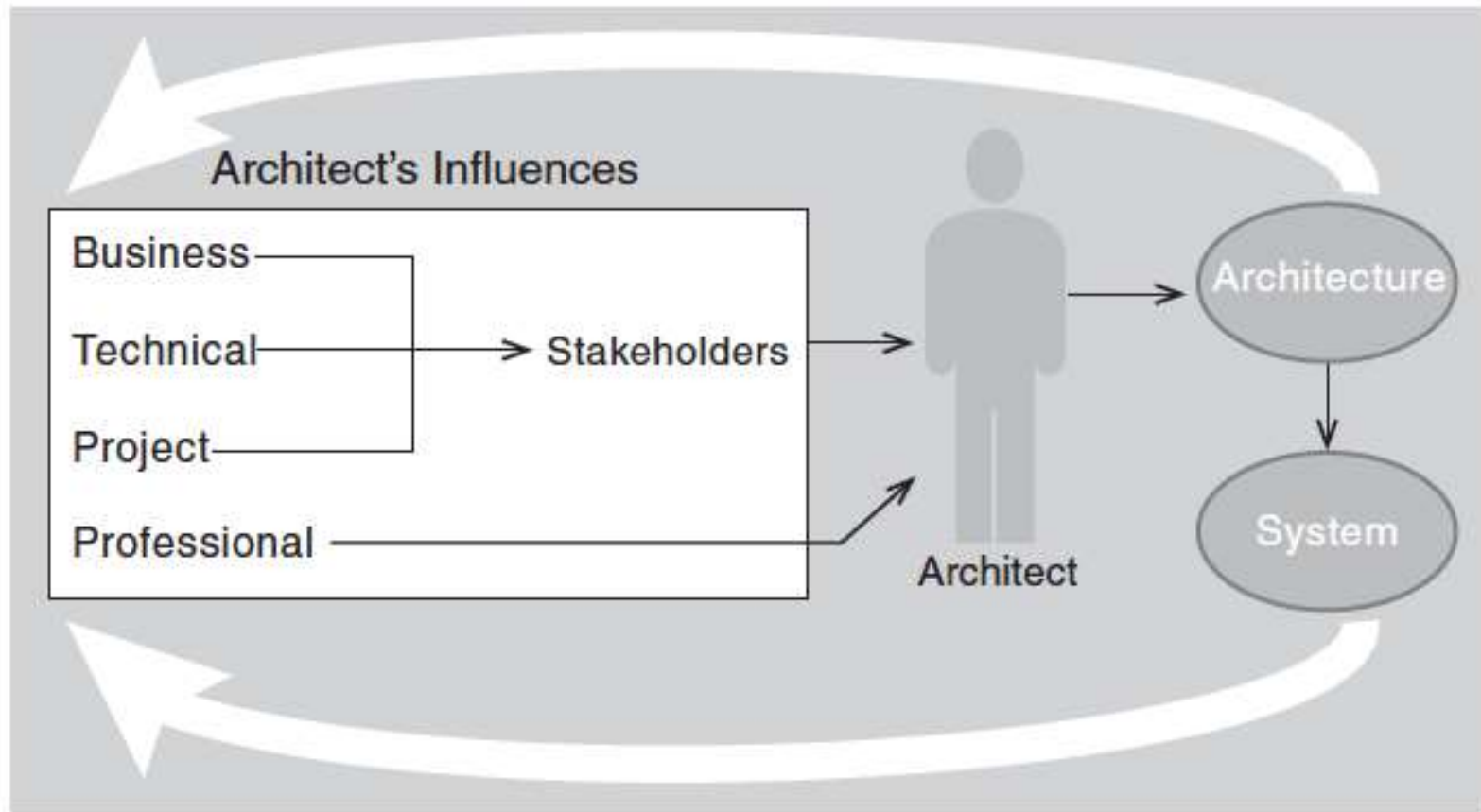


FIGURE 3.5 Architecture Influence Cycle

Summary



- Architectures exist in four different contexts.
 - Technical. The technical context includes the achievement of quality attribute requirements.
 - Project life cycle. Regardless of the software development methodology you use, you must perform specific activities.
 - Business. The system created from the architecture must satisfy the business goals of a wide variety of stakeholders.
 - Professional. You must have certain skills and knowledge to be an architect, and there are certain duties that you must perform as an architect.
- An architecture has influences that lead to its creation, and its existence has an impact on the architect, the organization, and, potentially, the industry.
- We call this cycle the Architecture Influence Cycle.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

What is Software Architecture

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs

What is Software Architecture



- What Software Architecture Is and What It Isn't
- Architectural Structures and Views
- Architectural Patterns
- What Makes a “Good” Architecture?

What is Software Architecture?



The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.

Architecture



◦Architecture is an abstraction:

- It omits information that has no ramifications outside of a single element.

◦Every system has an architecture:

- Even if it's not documented or is poorly designed.

◦Behavior is part of architecture:

- The definition includes "properties of both," which encompasses how elements interact at runtime.

Definition



This definition stands in contrast to other definitions that talk about the system's “early” or “major” design decisions.

- Many architectural decisions are made early, but not all are.
- Many decisions are made early that are not architectural.
- It's hard to look at a decision and tell whether or not it's “major.”

Structures, on the other hand, are fairly easy to identify in software, and they form a powerful tool for system design.

Architecture Is a Set of Software Structures



- A structure is a set of elements held together by a relation.
- Software systems are composed of many structures, and no single structure holds claim to being the architecture.
- There are three important categories of architectural structures.
 1. Module
 2. Component and Connector
 3. Allocation

Structures: three categories



◦Module Structures:

- How the system is partitioned as a set of code or data units.

◦Component-and-Connector (C&C) Structures:

- How the system is structured as a set of elements that have runtime behavior and interactions.

Allocation Structures:

- How the system relates to non-software structures (CPUs, file systems, teams).

Module Structures



- Some structures partition systems into implementation units, which we call modules.
- Modules are assigned specific computational responsibilities, and are the basis of work assignments for programming teams.
- In large projects, these elements (modules) are subdivided for assignment to sub-teams.

Component-and-connector Structures



- Other structures focus on the way the elements interact with each other at runtime to carry out the system's functions.
- We call runtime structures *component-and-connector (C&C) structures*.
- In our use, a component is always a runtime entity.
 - Suppose the system is to be built as a set of services.
 - The services, the infrastructure they interact with, and the synchronization and interaction relations among them form another kind of structure often used to describe a system.
 - These services are made up of (compiled from) the programs in the various implementation units – modules.

Allocation Structures



Allocation structures describe the mapping from software structures to the system's environments

- organizational
- developmental
- installation
- Execution

For example

- Modules are assigned to teams to develop, and assigned to places in a file structure for implementation, integration, and testing.
- Components are deployed onto hardware in order to execute.

Which Structures are Architectural?



- A structure is architectural if it supports reasoning about the system and the system's properties.
- The reasoning should be about an attribute of the system that is important to some stakeholder.

These include

- functionality achieved by the system
- the availability of the system in the face of faults
- the difficulty of making specific changes to the system
- the responsiveness of the system to user requests,
- many others.

Architecture is an Abstraction



- An architecture comprises software elements and how the elements relate to each other.
 - An architecture specifically omits certain information about elements that is not useful for reasoning about the system.
 - It omits information that has no ramifications outside of a single element.
 - An architecture selects certain details and suppresses others.
 - Private details of elements—details having to do solely with internal implementation—are not architectural.
- The architectural abstraction lets us look at the system in terms of its elements, how they are arranged, how they interact, how they are composed, what their properties are that support our system reasoning, and so forth.
- This abstraction is essential to taming the complexity of an architecture.
- We simply cannot, and do not want to, deal with all of the complexity all of the time.

Every System has a Software Architecture



- Every system comprises elements and relations among them to support some type of reasoning.
- But the architecture may not be known to anyone.
 - Perhaps all of the people who designed the system are long gone
 - Perhaps the documentation has vanished (or was never produced)
 - Perhaps the source code has been lost (or was never delivered)
- An architecture can exist independently of its description or specification.
- Documentation is critical.

Architecture Includes Behavior



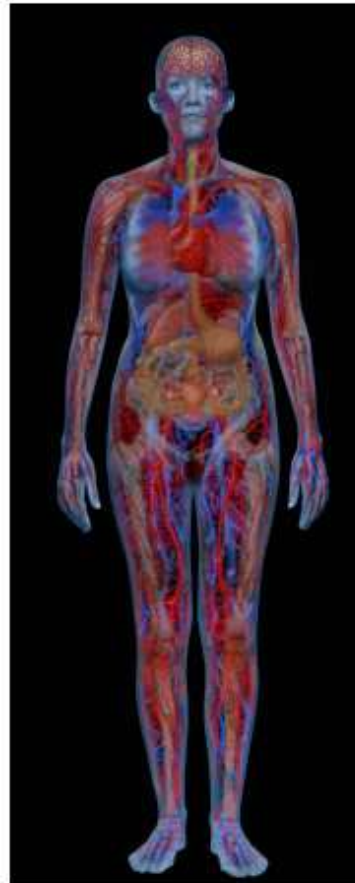
- The behavior of each element is part of the architecture insofar as that behavior can be used to reason about the system.
- This behavior embodies how elements interact with each other, which is clearly part of the definition of architecture.
- Box-and-line drawings that are passed off as architectures are not architectures at all.
 - When looking at the names of the a reader may well imagine the functionality and behavior of the corresponding elements.
 - But it relies on information that is not present – and could be wrong!
- This does not mean that the exact behavior and performance of every element must be documented in all circumstances.
 - Some aspects of behavior are fine-grained and below the architect's level of concern.
- To the extent that an element's behavior influences another element or influences the acceptability of the system as a whole, this behavior must be considered, and should be documented, as part of the software architecture.

Physiological Structures



- The neurologist, the orthopedist, the hematologist, and the dermatologist all have different views of the structure of a human body.
- Ophthalmologists, cardiologists, and podiatrists concentrate on specific subsystems.
- The kinesiologist and psychiatrist are concerned with different aspects of the entire arrangement's behavior.
- Although these views are pictured differently and have different properties, all are inherently related, interconnected.
- Together they describe the architecture of the human body.
- So it is with software!

Physiological Structures



Structures and Views



- A *view* is a representation of a coherent set of architectural elements, as written by and read by system stakeholders.
 - A view consists of a representation of a set of elements and the relations among them.
- A *structure* is the set of elements itself, as they exist in software or hardware.
- In short, a view is a representation of a structure.
 - For example, a module *structure* is the set of the system's modules and their organization.
 - A module *view* is the representation of that structure, documented according to a template in a chosen notation, and used by some system stakeholders.
- Architects design structures. They document views of those structures.

Module Structures



- Module structures embody decisions as to how the system is to be structured as a set of code or data units that have to be constructed or procured.
- In any module structure, the elements are modules of some kind (perhaps classes, or layers, or merely divisions of functionality, all of which are units of implementation).
- Modules are assigned areas of functional responsibility; there is less emphasis in these structures on how the software manifests at runtime.
- Module structures allow us to answer questions such as these:
 - What is the primary functional responsibility assigned to each module?
 - What other software elements is a module allowed to use?
 - What other software does it actually use and depend on?
 - What modules are related to other modules by generalization or specialization (i.e., inheritance) relationships?

Component-and-connector Structures



- Component-and-connector structures embody decisions as to how the system is to be structured as a set of elements that have runtime behavior (components) and interactions (connectors).
- Elements are runtime components such as services, peers, clients, servers, filters, or many other types of runtime element)
- Connectors are the communication vehicles among components, such as call-return, process synchronization operators, pipes, or others.
- Component-and-connector views help us answer questions such as these:
 - What are the major executing components and how do they interact at runtime?
 - What are the major shared data stores?
 - Which parts of the system are replicated?
 - How does data progress through the system?
 - What parts of the system can run in parallel?
 - Can the system's structure change as it executes and, if so, how?
- Component-and-connector views are crucially important for asking questions about the system's runtime properties such as performance, security, availability, and more.

Allocation structures



- Allocation structures show the relationship between the software elements and elements in one or more external environments in which the software is created and executed.
- Allocation views help us answer questions such as these:
 - What processor does each software element execute on?
 - In what directories or files is each element stored during development, testing, and system building?
 - What is the assignment of each software element to development teams?

Structures Provide Insight



- Structures play such an important role in our perspective on software architecture because of the analytical and engineering power they hold.
- Each structure provides a perspective for reasoning about some of the relevant quality attributes.
- For example:
 - The module structure, which embodies what modules use what other modules, is strongly tied to the ease with which a system can be extended or contracted.
 - The concurrency structure, which embodies parallelism within the system, is strongly tied to the ease with which a system can be made free of deadlock and performance bottlenecks.
 - The deployment structure is strongly tied to the achievement of performance, availability, and security goals.
 - And so forth.

Some Useful Module Structures



Decomposition structure

- The units are modules that are related to each other by the *is-a-submodule-of* relation.
- It shows how modules are decomposed into smaller modules recursively until the modules are small enough to be easily understood.
- Modules often have products (such as interface specifications, code, test plans, etc.) associated with them.
- The decomposition structure determines, to a large degree, the system's modifiability, by assuring that likely changes are localized.
- This structure is often used as the basis for the development project's organization, including the structure of the documentation, and the project's integration and test plans.
- The units in this structure tend to have names that are organization-specific such as "segment" or "subsystem."

Some Useful Module Structures



Uses structure.

- The units here are also modules, perhaps classes.
- The units are related by the *uses* relation, a specialized form of dependency.
- A unit of software *uses* another if the correctness of the first requires the presence of a correctly functioning version (as opposed to a stub) of the second.
- The uses structure is used to engineer systems that can be extended to add functionality, or from which useful functional subsets can be extracted.
- The ability to easily create a subset of a system allows for incremental development.

Some Useful Module Structures



Layer structure

- The modules in this structure are called *layers*.
- A layer is an abstract “virtual machine” that provides a cohesive set of services through a managed interface.
- Layers are *allowed to use* other layers in a strictly managed fashion.
 - In strictly layered systems, a layer is only allowed to use a single other layer.
- This structure imbues a system with portability, the ability to change the underlying computing platform.

Some Useful Module Structures



- Class (or generalization) structure
 - The module units in this structure are called *classes*.
 - The relation is *inherits from* or *is an instance of*.
 - This view supports reasoning about collections of similar behavior or capability
 - e.g., the classes that other classes inherit from and parameterized differences
 - The class structure allows one to reason about reuse and the incremental addition of functionality.
 - If any documentation exists for a project that has followed an object-oriented analysis and design process, it is typically this structure.

Some Useful Module Structures



Data model

- The data model describes the static information structure in terms of data entities and their relationships.
 - For example, in a banking system, entities will typically include Account, Customer, and Loan.
 - Account has several attributes, such as account number, type (savings or checking), status, and current balance.

Some Useful C&C Structures



- The relation in all component-and-connector structures is attachment, showing how the components and the connectors are hooked together.
- The connectors can be familiar constructs such as “invokes.”
- Useful C&C structures include:
 - Service structure
 - The units are services that interoperate with each other by service coordination mechanisms such as SOAP.
 - The service structure helps to engineer a system composed of components that may have been developed anonymously and independently of each other.
 - Concurrency structure
 - This structure helps determine opportunities for parallelism and the locations where resource contention may occur.
 - The units are components
 - The connectors are their communication mechanisms.
 - The components are arranged into logical threads.

Some Useful Allocation Structures



Deployment structure

- The deployment structure shows how software is assigned to hardware processing and communication elements.
- The elements are software elements (usually a process from a C&C view), hardware entities (processors), and communication pathways.
- Relations are allocated-to, showing on which physical units the software elements reside, and migrates-to if the allocation is dynamic.
- This structure can be used to reason about performance, data integrity, security, and availability.
- It is of particular interest in distributed and parallel systems.

Some Useful Allocation Structures



Implementation structure

- This structure shows how software elements (usually modules) are mapped to the file structure(s) in the system's development, integration, or configuration control environments.
- This is critical for the management of development activities and build processes.

Work assignment structure

- This structure assigns responsibility for implementing and integrating the modules to the teams who will carry it out.
- Having a work assignment structure be part of the architecture makes it clear that the decision about who does the work has architectural as well as management implications.
- The architect will know the expertise required on each team.
- This structure will also determine the major communication pathways among the teams: regular teleconferences, wikis, email lists, and so forth.

Relating Structures to Each Other



- Elements of one structure will be related to elements of other structures, and we need to reason about these relations.
 - A module in a decomposition structure may be manifested as one, part of one, or several components in one of the component-and-connector structures.
- In general, mappings between structures are many to many.

Modules vs. Components

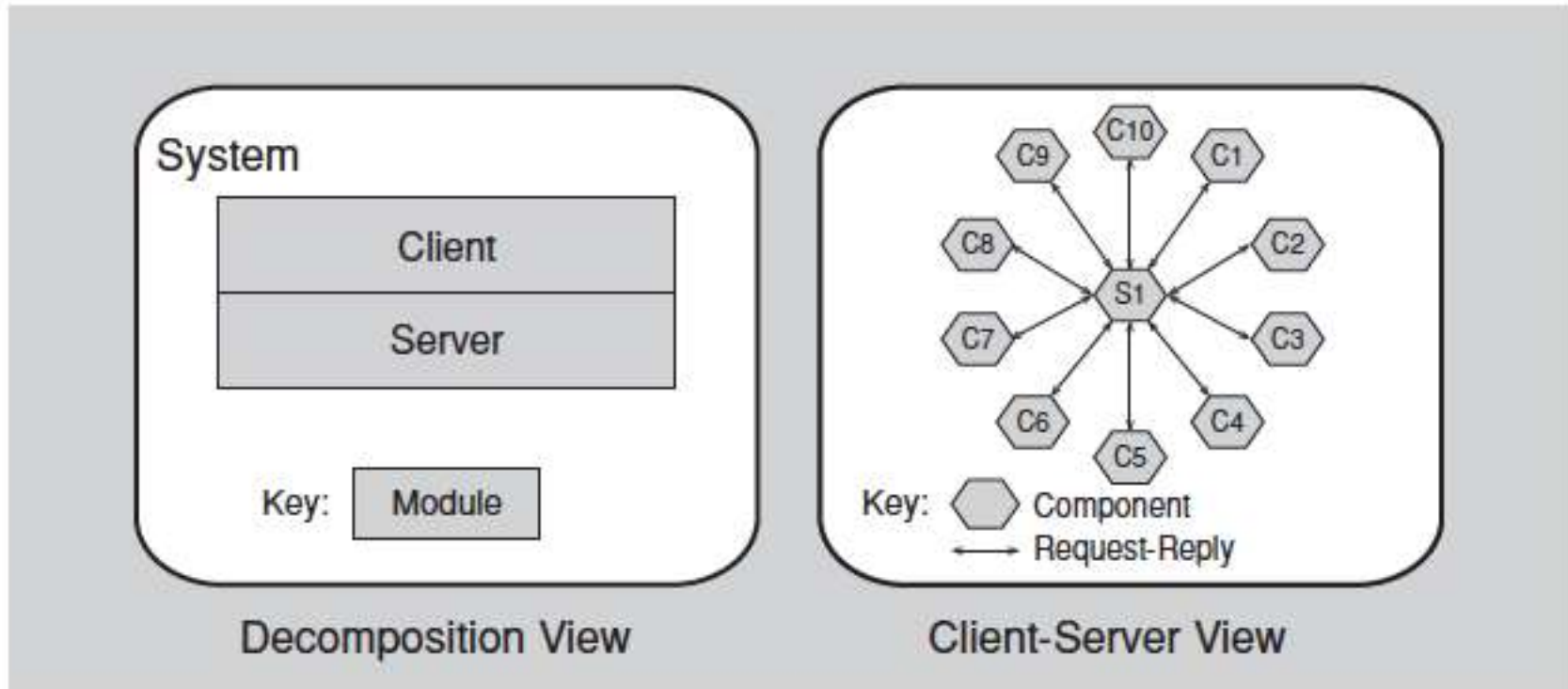


FIGURE 1.2 Two views of a client-server system

Architectural Patterns



- Architectural elements can be composed in ways that solve particular problems.
 - The compositions have been found useful over time, and over many different domains
 - They have been documented and disseminated.
 - These compositions of architectural elements, called architectural patterns.
 - Patterns provide packaged strategies for solving some of the problems facing a system.
- An architectural pattern delineates the element types and their forms of interaction used in solving the problem.
- A common module type pattern is the Layered pattern.
 - When the uses relation among software elements is strictly unidirectional, a system of layers emerges.
 - A layer is a coherent set of related functionality.
 - Many variations of this pattern, lessening the structural restriction, occur in practice.

Architectural Patterns



Common component-and-connector type patterns:

- **Shared-data (or repository) pattern.**
 - This pattern comprises components and connectors that create, store, and access persistent data.
 - The repository usually takes the form of a (commercial) database.
 - The connectors are protocols for managing the data, such as SQL.
- **Client-server pattern.**
 - The components are the clients and the servers.
 - The connectors are protocols and messages they share among each other to carry out the system's work.

Architectural Patterns



Common allocation patterns:

- Multi-tier pattern
 - Describes how to distribute and allocate the components of a system in distinct subsets of hardware and software, connected by some communication medium.
 - This pattern specializes the generic deployment (software-to-hardware allocation) structure.
- Competence center pattern and platform pattern
 - These patterns specialize a software system's work assignment structure.
 - In competence center, work is allocated to sites depending on the technical or domain expertise located at a site.
 - In platform, one site is tasked with developing reusable core assets of a software product line, and other sites develop applications that use the core assets.

What Makes a “Good” Architecture?



- There is no such thing as an inherently good or bad architecture.
- Architectures are either more or less fit for some purpose
- Architectures can be evaluated but only in the context of specific stated goals.
- There are, however, good rules of thumb.

Rule of Thumb



Process Rules:

- ⑩ The architecture should be the product of a **single architect** or a small group with an identified leader.
- ⑩ The architect should have a prioritized list of **quality attribute requirements**.
- ⑩ The architecture should be **documented** using views and **evaluated** using a formal method (like ATAM).

Product Rules:

- ⑩ Use **information hiding** and separation of concerns.
- ⑩ Achieve quality attributes using well-known **tactics**.
- ⑩ Never depend on a particular version of a commercial tool.
- ⑩ Minimize the number of ways components interact.

Structural “Rules of Thumb”



- The architecture should be the product of a single architect or a small group of architects with an identified technical leader.
 - This approach gives the architecture its conceptual integrity and technical consistency.
 - This recommendation holds for Agile and open source projects as well as “traditional” ones.
 - There should be a strong connection between the architect(s) and the development team.
- The architect (or architecture team) should base the architecture on a prioritized list of well-specified quality attribute requirements.
- The architecture should be documented using views. The views should address the concerns of the most important stakeholders in support of the project timeline.
- The architecture should be evaluated for its ability to deliver the system’s important quality attributes.
 - This should occur early in the life cycle and repeated as appropriate.
- The architecture should lend itself to incremental implementation,
 - Create a “skeletal” system in which the communication paths are exercised but which at first has minimal functionality.

Structural “Rules of Thumb”



- The architecture should feature well-defined modules whose functional responsibilities are assigned on the principles of information hiding and separation of concerns.
 - The information-hiding modules should encapsulate things likely to change
 - Each module should have a well-defined interface that encapsulates or “hides” the changeable aspects from other software
- Unless your requirements are unprecedented your quality attributes should be achieved using well-known architectural patterns and tactics specific to each attribute.
- The architecture should never depend on a particular version of a commercial product or tool. If it must, it should be structured so that changing to a different version is straightforward and inexpensive.
- Modules that produce data should be separate from modules that consume data.
 - This tends to increase modifiability
 - Changes are frequently confined to either the production or the consumption side of data.

Structural “Rules of Thumb”



- Don't expect a one-to-one correspondence between modules and components.
- Every process should be written so that its assignment to a specific processor can be easily changed, perhaps even at runtime.
- The architecture should feature a small number of ways for components to interact.
 - The system should do the same things in the same way throughout.
 - This will aid in understandability, reduce development time, increase reliability, and enhance modifiability.
- The architecture should contain a specific (and small) set of resource contention areas, the resolution of which is clearly specified and maintained.

Summary



- The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.
- A structure is a set of elements and the relations among them.
- A view is a representation of a coherent set of architectural elements. A view is a representation of one or more structures.

Summary



- There are three categories of structures:
 - Module structures show how a system is to be structured as a set of code or data units that have to be constructed or procured.
 - Component-and-connector structures show how the system is to be structured as a set of elements that have runtime behavior (components) and interactions (connectors).
 - Allocation structures show how the system will relate to nonsoftware structures in its environment (such as CPUs, file systems, networks, development teams, etc.).
- Structures represent the primary engineering leverage points of an architecture.
- Every system has a software architecture, but this architecture may be documented and disseminated, or it may not be.
- There is no such thing as an inherently good or bad architecture. Architectures are either more or less fit for some purpose.